OS – ORANGE PROBLEM 2

KRISHITHA KANKANALA

PES2UG24CS234

SEM 4 – D SECTION

GUTHUB REPO: https://github.com/krista9669/PES2UG24CS234-pes-vcs

Phase 1:

A)

```
krishitha@krista:~$ cd PES2UG24CS234-pes-vcs
krishitha@krista:~/PES2UG24CS234-pes-vcs$ ls
Makefile   commit.c  commit.o  index.h  object.c  pes    pes.h  test_objects    test_objects.o    test_tree    test_tree.o  tree.h
README.md  commit.h  index.c   index.o  object.o  pes.c  pes.o  test_objects.c  test_sequence.sh  test_tree.c  tree.c       tree.o
krishitha@krista:~/PES2UG24CS234-pes-vcs$ make all
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function 'object_read':
object.c:118:5: warning: ignoring return value of 'fread' declared with attribute 'warn_unused_result' [-Wunused-result]
  118 |     fread(buffer, 1, size, f);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
gcc -o test_objects test_objects.o object.o -lcrypto
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
krishitha@krista:~/PES2UG24CS234-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

B)

```
krishitha@krista:~/PES2UG24CS234-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

Phase 2:

A)

```
krishitha@krista:~/PES2UG24CS234-pes-vcs$ make test_tree
make: 'test_tree' is up to date.
krishitha@krista:~/PES2UG24CS234-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

B)

```
krishitha@krista:~/PES2UG24CS234-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
krishitha@krista:~/PES2UG24CS234-pes-vcs$ xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf | head -
20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
```

Phase 3:

A)

```
krishitha@krista:~/PES2UG24CS234-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
krishitha@krista:~/PES2UG24CS234-pes-vcs$ echo "hello" > file1.txt
krishitha@krista:~/PES2UG24CS234-pes-vcs$ echo "world" > file2.txt
krishitha@krista:~/PES2UG24CS234-pes-vcs$ ./pes add file1.txt file2.txt
krishitha@krista:~/PES2UG24CS234-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  object.c
  untracked:  commit.c
  untracked:  commit.h
  untracked:  test_objects
  untracked:  .gitignore
  untracked:  pes.c
  untracked:  README.md
  untracked:  tree.c
  untracked:  pes.h
  untracked:  test_tree.c
  untracked:  test_tree
  untracked:  tree.h
  untracked:  file.txt
  untracked:  Makefile
  untracked:  test_sequence.sh
  untracked:  index.c
  untracked:  test_objects.c
  untracked:  index.h
```

B)

```
krishitha@krista:~/PES2UG24CS234-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776443122 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776443129 6 file2.txt
```

Phase 4:

A)

```
krishitha@krista:~/PES2UG24CS234-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
krishitha@krista:~/PES2UG24CS234-pes-vcs$ echo "Hello" > hello.txt
krishitha@krista:~/PES2UG24CS234-pes-vcs$ ./pes add hello.txt
krishitha@krista:~/PES2UG24CS234-pes-vcs$ ./pes commit -m "Initial commit"
Committed: 8145fa7b3028... Initial commit
krishitha@krista:~/PES2UG24CS234-pes-vcs$ echo "World" >> hello.txt
krishitha@krista:~/PES2UG24CS234-pes-vcs$ ./pes add hello.txt
krishitha@krista:~/PES2UG24CS234-pes-vcs$ ./pes commit -m "Add world"
Committed: 2f8766d011b7... Add world
krishitha@krista:~/PES2UG24CS234-pes-vcs$ echo "Bye" > bye.txt
krishitha@krista:~/PES2UG24CS234-pes-vcs$ ./pes add bye.txt
krishitha@krista:~/PES2UG24CS234-pes-vcs$ ./pes commit -m "Add farewell"
Committed: 80d9c787487c... Add farewell
krishitha@krista:~/PES2UG24CS234-pes-vcs$ ./pes log
commit 80d9c787487c2373f82c6932ccf96f3a327018752e9c36d88b2d4e7fb3b2d224
Author: PES User <pes@localhost>
Date:   1776443405

    Add farewell

commit 2f8766d011b7633b0707cdff6034b3a378540e84bcd5dd533f7bdc05389561ff
Author: PES User <pes@localhost>
Date:   1776443394

    Add worldell

commit 8145fa7b30282b05099bc34dd68f39f8842702f6dd513d5a3db63811813f18e0
Author: PES User <pes@localhost>
Date:   1776443382

    Initial committial commiQ
```

B)

```
krishitha@krista:~/PES2UG24CS234-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2f/8766d011b7633b0707cdff6034b3a378540e84bcd5dd533f7bdc05389561ff
.pes/objects/48/1407f7237f7bd749dca31e2d8422bafe211741d5556f007e0faf821ffc7179
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/80/d9c787487c2373f82c6932ccf96f3a327018752e9c36d88b2d4e7fb3b2d224
.pes/objects/81/45fa7b30282b05099bc34dd68f39f8842702f6dd513d5a3db63811813f18e0
.pes/refs/heads/main
```

C)

```
krishitha@krista:~/PES2UG24CS234-pes-vcs$ cat .pes/refs/heads/main
80d9c787487c2373f82c6932ccf96f3a327018752e9c36d88b2d4e7fb3b2d224
krishitha@krista:~/PES2UG24CS234-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Phase 5 and 6 (Analysis-Only Questions):

Q5.1) pes checkout <branch> updates .pes/HEAD to point to the branch and reads the commit hash from .pes/refs/heads/<branch>. The working directory is then updated to match the commit's tree by adding, modifying, or deleting files. The operation is complex because it must safely update files without overwriting uncommitted changes.

Q5.2) Compare working directory files with the index:

- missing file → deleted
- different size/mtime or hash → modified

If a file is modified locally and differs in the target branch, checkout must refuse. This detects a dirty working directory.

Q5.3) In detached HEAD, commits are not linked to any branch. They become unreachable if you switch branches. To recover, create a branch at that commit or use reflog to find it.

Q6.1) Start from all branch heads and traverse commits, trees, and blobs, marking reachable objects using a hash set. Delete objects not in the set. For 100k commits, about 100k objects (plus trees/blobs) are visited once.

Q6.2) GC can delete objects that a commit is about to reference if run concurrently. Git avoids this using atomic writes, updating objects before references, and protecting recent objects from deletion.